

**MATERIAL DE APOYO INFORME
PRELIMINAR**

DESAFIO I: SWEET CRUSH

CONTEXTUALIZACIÓN, ANALISIS Y DISEÑO

Informática II – Semestre 2026-2

JOSE GABRIEL GIRALDO FLOREZ

September 2026

1. Contextualización

1.1 Descripción general del desafío

El Desafío I tiene como base el desarrollo, diseño e implementación del juego Sweet Crush con el uso del lenguaje C++ dentro del framework QT, bajo restricciones e instrucciones estrictas de programación estructurada: sin el uso de struct, class, template, ni STL. Debe tenerse muy en cuenta la representación y manipulación del tablero ya que cada ficha del tablero debe almacenarse usando exactamente 3 bits para su representación, empaquetados de forma continua en un bloque de memoria dinámica (Uso obligatorio), sin relleno para alinear cada ficha a un byte.

1.2 Restricciones y condicionamiento del diseño

- No uso de struct, class, template.
- No STL (nada de vector, string, algorithm, map, etc.)
- No usar objetos tipo string.
- No sintaxis ANSI C pura.
- Implementación obligatoria en QT.
- Uso de representación de las fichas de 3 bits, sin relleno.
- El tablero siempre se mantiene en bits; nunca se copia en un arreglo normal.
- Gestión “real” de memoria (debe crecer o decrecer según sus dimensiones, no mantiene una dimensión estática.)
- Uso de programación modular/multi-archivos.

Dichas restricciones corresponden de forma directa a los tres criterios de evaluación propuestos para el desafío (30 % operadores de bits, 35 % gestión de memoria, 35 % lógica de juego).

2. Análisis del problema

2.1 Modelo de datos: representación de fichas

Se pide la implementación de 6 tipos de fichas, con una representación de 3 bits, generando 8 combinaciones posibles. Quedando 2 combinaciones libres para estados adicionales (como estado vacío y especial):

000	FICHA_A
001	FICHA_B
010	FICHA_C
011	FICHA_D
100	FICHA_E
101	FICHA_F
110	VACIO (Posición libre, resultado de una eliminación.)
111	ESPECIAL (A decisión propia (opcional): puede quedar sin usar o mejoras adicionales del ju

2.2 Modelo de datos: representación del tablero

El tablero debe ser reservado con una secuencia continua de bits, sin ningún tipo de relleno entre fichas. Permitiendo así un tablero de f filas y c columnas.

Posible implementación:

```
pos = fila * columnas + columna
bit_inicial = pos * 3
byte = bit_inicial / 8
desfase = bit_inicial % 8
bits_necesarios = 3 * filas * columnas
bytes_necesarios = techo (bits_necesarios / 8)
```

Ya que cada ficha ocupa 3 bits y cada byte 8, las fronteras entre las fichas no coinciden necesariamente con la frontera entre bytes, por lo tanto, una ficha puede quedar completamente contenida en un byte, o repartida entre 2 bytes consecutivos. El análisis debe tomar ambos casos de forma unificada.

Estrategia de lectura/escritura sin ramificar en casos:

En lugar de tomar por separado fichas en 1 solo byte y fichas que se encuentran repartidas en 2 bytes, se implementara una combinación entre el byte actual y el siguiente en una ventana de 16 bits:

```
Ventana = tablero[byte] | (tablero[byte + 1] << 8)
Ficha = (ventana >> desfase) & 0b111
```

Se podría implementar sin necesidad de condicionales, para los 3 escenarios posibles, ya que, con el corrimiento y la máscara se extraen los 3 bits correctos independientemente del cruce de fronteras de bytes. Es esencial tener cuidado para no leer byte + 1 cuando no exista (Esto solo se da en caso que la última ficha del tablero caiga exactamente dentro de un único byte, con desfase ≤ 5). Para su escritura se usa el mismo principio donde se limpia la ficha anterior con una máscara negada sobre la ventana de 166 bits, insertando el nuevo valor con su corrimiento, y separando el resultado en uno o dos bytes según el caso.

2.3 Análisis de gestión de memoria

Se pide que la memoria reservada corresponda siempre a las dimensiones actuales del tablero, teniendo además una excepción explícita en el documento: “Al eliminar una fila o columna, la reasignación física de memoria solo debe ocurrir cuando la utilización caiga por debajo del 65 % respecto a la capacidad actualmente reservada”.

Posible implementación:

```
uso = bits_necesarios_nuevo / (bytes_asignados_actuales * 8)

si uso < 0.65:
    reasignar el buffer a bytes_necesarios (bytes_necesarios_nuevo)
si no:
    conservar el buffer actual (queda con bytes sobrantes, sin usar, al
    final)
```

Así, cuando se reasigna, los bits válidos siguen comenzando en el bit 0 del byte 0, porque los índices siempre se calculan desde 0, por su parte los bits no utilizados quedan agrupados al final del arreglo, cumpliendo la exigencia del enunciado de: “Las tramas de bits válidos siempre deben iniciar en el bit menos significativo... los bits inválidos deben agruparse a la izquierda de toda la trama”.

Del mismo modo esto evita reasignaciones costosas de memoria cuando el usuario decida agregar y eliminar filas o columnas repetidamente en tamaños cercanos.

2.4 Análisis de operaciones para agregar/eliminar filas o columnas

Dado que las fichas no están alineadas por fila, si se agrega o quito una fila o columna en la mitad del tablero, casi todas las fichas que están después se mueven de lugar en la memoria. Por eso no se intenta mover los bits a mano caso por caso, sino que se armara el tablero nuevo desde cero:

1. Calculo cuántas filas y columnas va a tener el tablero nuevo, y con eso cuánta memoria necesito (si agrego, reservo justo lo necesario; si elimino, aplico la regla del 65 %).
2. Se reserve la nueva memoria.
3. Recorre el tablero nuevo posición por posición: si esa posición ya existía en el tablero antiguo, se copia la ficha; si es una posición nueva (por la fila o columna agregada), se pone una ficha generada al azar.
4. Se hace liberación del tablero viejo y actualizamos filas, columnas y el tamaño en memoria.

Así se sigue usando las funciones de extraer e insertar bits en cada paso (que es justamente lo que se pide en el ejercicio), sin preocuparse por programar a mano todos los diferentes casos, cuando una ficha queda partida entre dos bytes, resolviéndolo con las mismas funciones.

2.5 Análisis de detección de combinaciones y cascada

Para las combinaciones es necesario la corrida de 3 o más fichas iguales, consecutivas ya sea en sentido vertical o horizontal. Se pide que una ficha la cual haga parte de una combinación horizontal y vertical se debe procesar únicamente una vez, lo cual requiere marcar antes de eliminar, en lugar de eliminar directamente durante la detección.

En otro punto, una cascada se da tras una eliminación y reorganización consecuente, es decir gracias a la gravedad y el relleno de las celdas, se generan nuevas combinaciones sin intervención del jugador. Dicho proceso debe darse hasta que no se generen aquellas combinaciones de forma automática, debido al fenómeno presente.

2.6 Problemas de desarrollo previstos

Esta sección permite como análisis previo anticipar los puntos de mayor riesgo en la ejecución e implementación del programa y su correcto funcionamiento dando tareas necesarias como:

- Verificación de la extracción e inserción de bits exactamente en los índices que crucen fronteras entre bytes.
- Corrección en el calculo de la regla del 65 % en escenarios de eliminación sucesiva de filas o columnas.
- Verificación de que una ficha en un cruce horizontal – vertical se elimine una única vez y se cuente de forma correcta en las estadísticas.
- Consistencia al insertar o eliminar filas o columnas en las posiciones intermedias (no los para los extremos).

3. Diseño de solución

3.1 Arquitectura propuesta de módulos

Como requisito el proyecto será organizado en módulos .h / .cpp independientes, con cada uno correspondiente a una función explícita del enunciado o de los criterios de evaluación para el correcto funcionamiento e implementación del programa.

Modelo base:

- Main.cpp
- bits.h / bits.cpp
- tablero / tablero.cpp (creación – liberación del tablero, obtener – establecer fichas.)
- fichas.h / fichas.cpp (códigos, generaciones aleatorias, símbolos visuales)
- combinaciones.h / combinaciones.cpp (detecciones horizontales/verticales, marcado, eliminación.)
- reorganización.h / reorganización.cpp (gravedad, relleno, bucle de cascada.)
- estructura.h / estructura.cpp (agregar/eliminar filas o columnas, regla del 65 %.)

- estado.h / estado.cpp (contador de partida y puntuación)
- visualización.h / visualización.cpp (mostrar tablero en fichas y en binario, estadísticas.)

3.2 Funciones por modulo

Posibles desarrollos:

bits.h / bits.cpp – Primitivas genéricas de bits

```
unsigned char extraerBits3 (const unsigned char* buffer, int bitInicial);  
void insertarBits3 (unsigned char* buffer, int bitInicial, unsigned char  
    valor);  
int bitsNecesarios (int filas, int columnas);  
int bytesNecesarios (int bits);
```

Estas son las funciones base del proyecto que permite acceder directamente al arreglo de bytes del tablero; todos los módulos pasan por dichas funciones.

tablero.h / tablero.cpp

```
void crearTablero (unsigned char** tablero, int filas, int columnas, int*  
    bytesAsignados);  
void liberarTablero(unsigned char** tablero);  
unsigned char obtenerFicha(const unsigned char* tablero, int columnas, int  
    fila, int columna);  
void establecerFicha(unsigned char* tablero, int columnas, int fila, int  
    columna, unsigned char valor);  
void inicializarTableroAleatorio(unsigned char* tablero, int filas, int  
    columnas);
```

fichas.h / fichas.cpp

```
unsigned char generarFichaAleatoria(); // usa <random>  
char simboloFicha(unsigned char codigo); // 'A'..'F' o '.' para vaco  
void inicializarGeneradorAleatorio();
```

combinaciones.h / combinaciones.cpp

```
int detectarHorizontales(const unsigned char* tablero, int filas, int
    columnas, unsigned char* marcador);
int detectarVerticales(const unsigned char* tablero, int filas, int
    columnas, unsigned char* marcador);
int eliminarMarcadas(unsigned char* tablero, int filas, int columnas, const
    unsigned char* marcador);
```

Nota: El arreglo marcador será una estructura auxiliar temporal (no una copia de las fichas): tomando un byte por posición indicada si está marcada para su eliminación. Ambas funciones de detección están pensadas para hacer un OR sobre el mismo arreglo, de modo que una ficha en cruce horizontal – vertical quede marcada 1 sola vez.

reorganizacion.h / reorganizacion.cpp

```
void aplicarGravedad(unsigned char* tablero, int filas, int columnas);
void rellenarVacios(unsigned char* tablero, int filas, int columnas);
int procesarCascadas(unsigned char* tablero, int filas, int columnas, int*
    fichasEliminadas, int* combinacionesDetectadas);
```

estructura.h / estructura.cpp

```
void agregarFila(unsigned char** tablero, int* filas, int columnas, int*
    bytesAsignados, int posicion);
void eliminarFila(unsigned char** tablero, int* filas, int columnas, int*
    bytesAsignados, int posicion);
void agregarColumna(unsigned char** tablero, int filas, int* columnas, int*
    bytesAsignados, int posicion);
void eliminarColumna(unsigned char** tablero, int filas, int* columnas, int
    * bytesAsignados, int posicion);
int decidirRedimensionar(int bitsNecesarios, int bytesAsignadosActuales);
// Para regla del 65%
```

estado.h / estado.cpp

El estado de la partida se manejará como un módulo con variables internas y funciones de acceso.


```
void inicializarEstado();
void registrarEliminacionManual();
void registrarFichasEliminadas(int cantidad);
void registrarCombinacion();
void registrarCascadas(int cantidad);
void sumarPuntuacion(int puntos);
int obtenerEliminaciones();
int obtenerFichasEliminadas();
int obtenerCombinaciones();
int obtenerCascadas();
int obtenerPuntuacion();
```

visualizacion.h / visualizacion.cpp

```
void mostrarTableroFichas(const unsigned char* tablero, int filas, int
    columnas);
void mostrarTableroBinario(const unsigned char* tablero, int filas, int
    columnas, int bytesAsignados);
void mostrarEstadisticas();
```

3.3 Criterio de puntuación

- Eliminación manual sin combo resultante: 0 puntos.
- Cada ficha eliminada dentro de una combinación: 10 puntos.
- Multiplicador por nivel de cascada (premia reacciones en cadena): cascada 1 → $\times 1$, cascada 2 → $\times 2$, cascada 3 → $\times 3$, etc.

Nota importante

Nota importante: Esto es un diseño base con todas las posibles implementaciones dentro del programa y algunas propuestas tanto de módulos como funciones para un desarrollo ordenado del mismo, teniendo en cuenta cada una de las restricciones y exigencias del trabajo, por ende, es posible en el avance y creación del programa la implementación de funciones extra no documentadas para esta versión del análisis y desarrollo.

Att: Jose Gabriel Giraldo Flórez